Homework 3

PSTAT 131/231 Elliott Liu

Contents

Binary Classification

For this assignment, we will be working with part of a Kaggle data set that was the subject of a machine learning competition and is often used for practicing ML models. The goal is classification; specifically, to predict which passengers would survive the Titanic shipwreck.

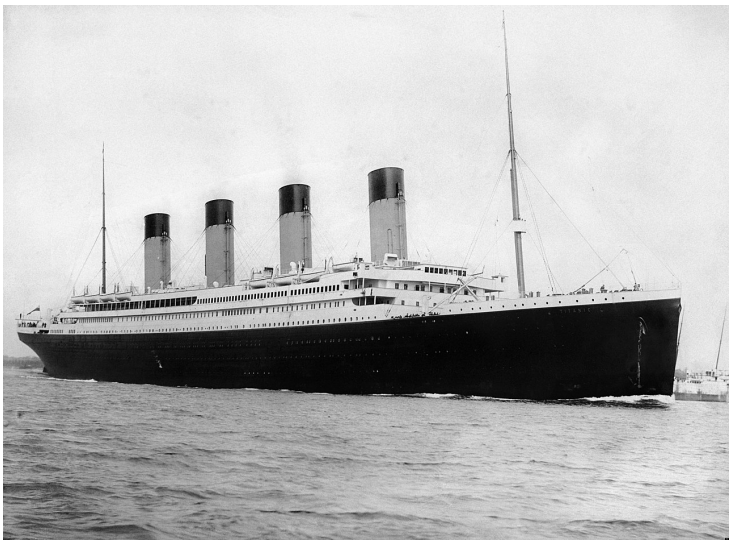


Figure 1: Fig. 1: RMS Titanic departing Southampton on April 10, 1912.

Load the data from `data/titanic.csv` into *R* and familiarize yourself with the variables it contains using the codebook (`data/titanic_codebook.txt`).

```
library(tidyverse)
library(tidymodels)
library(corrplot)
library(poissonreg)
tidymodels_prefer()
set.seed(131)
```

Notice that `survived` and `pclass` should be changed to factors. When changing `survived` to a factor, you may want to reorder the factor so that *"Yes"* is the first level.

```
df <- read.csv("titanic.csv") %>% mutate(survived = factor(survived, levels = c ("Yes", "No")), pclass =
```

Make sure you load the `tidyverse` and `tidymodels`!

Remember that you'll need to set a seed at the beginning of the document to reproduce your results.

Question 1

Split the data, stratifying on the outcome variable, `survived`. You should choose the proportions to split the data into. Verify that the training and testing data sets have the appropriate number of observations. Take a look at the training data and note any potential issues, such as missing data.

Why is it a good idea to use stratified sampling for this data?

```
data_split <- initial_split(
  df,
  prop = 0.75,
  strata = survived
)

train_data <- training(data_split)
test_data <- testing(data_split)
```

```
total <- dim(df)
total
```

```
## [1] 891  12
```

```
train_num <- dim(train_data)
train_num
```

```
## [1] 667  12
```

```
test_num <- dim(test_data)
test_num
```

```
## [1] 224  12
```

```
train_num[1] / total[1]
```

```
## [1] 0.7485971
```

```
test_num[1] / total[1]
```

```
## [1] 0.2514029
```

The training data has 667 observations, and testing data has 224 observations.

```
train_data %>% summary()
```

```
##   passenger_id   survived  pclass      name               sex
##  Min.   :  1.0   Yes:256   1:163   Length:667         Length:667
##  1st Qu.:225.5   No :411   2:136   Class :character   Class :character
##  Median :436.0             3:368   Mode  :character   Mode  :character
##  Mean   :445.1
##  3rd Qu.:673.5
##  Max.   :890.0
##
##       age            sib_sp           parch           ticket
##  Min.   : 0.42   Min.   :0.0000   Min.   :0.0000   Length:667
##  1st Qu.:20.00   1st Qu.:0.0000   1st Qu.:0.0000   Class :character
##  Median :28.00   Median :0.0000   Median :0.0000   Mode  :character
##  Mean   :29.62   Mean   :0.5307   Mean   :0.3808
##  3rd Qu.:38.00   3rd Qu.:1.0000   3rd Qu.:0.0000
##  Max.   :80.00   Max.   :8.0000   Max.   :6.0000
##  NA's   :139
##       fare            cabin             embarked
##  Min.   :  0.000   Length:667         Length:667
##  1st Qu.:  7.896   Class :character   Class :character
##  Median : 14.458   Mode  :character   Mode  :character
##  Mean   : 33.029
##  3rd Qu.: 30.285
##  Max.   :512.329
##
```

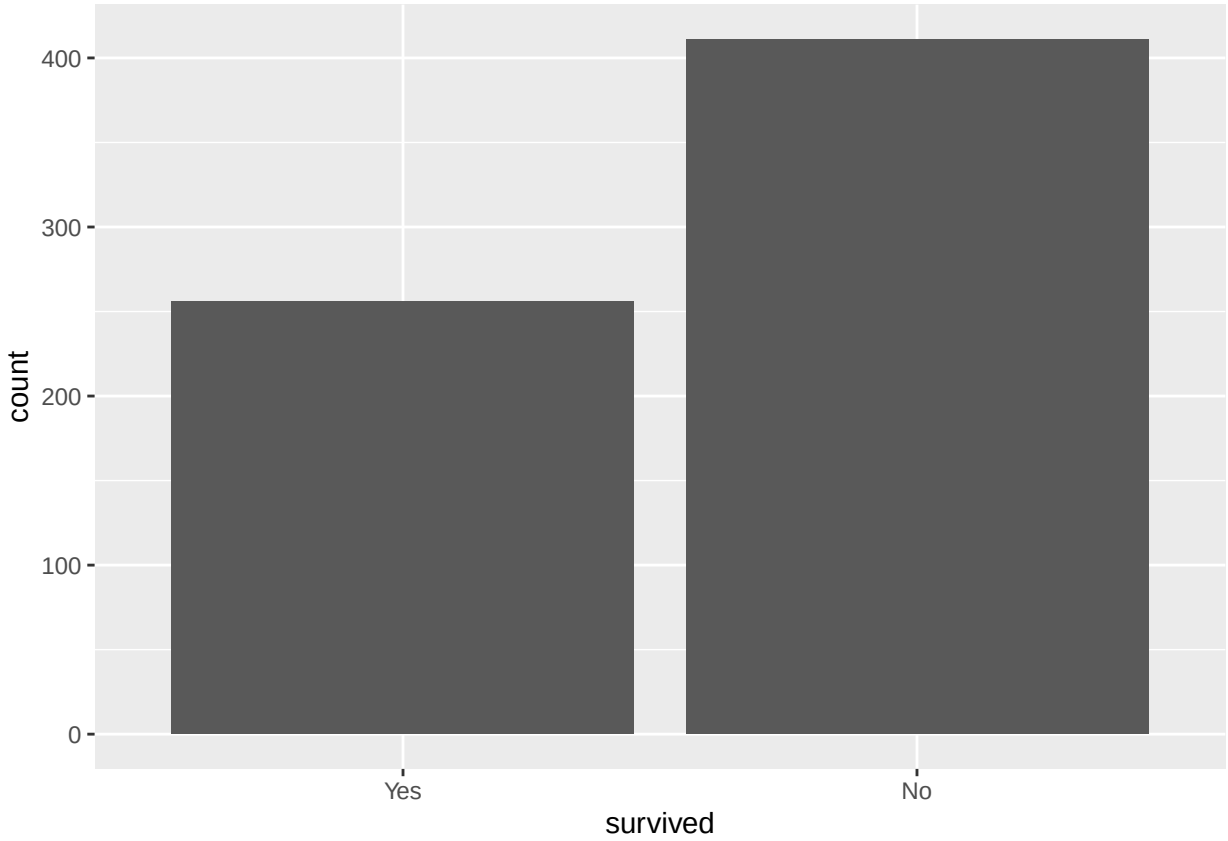
The training dataset contains 139 missing values in the age variable.

Stratified sampling is a good idea in this case because it can let both the training and testing datasets have the original proportion of survivors and non-survivors. Because the outcome variable is a little bit imbalanced; therefore, a stratification method can help avoid biased model training and evaluation. Also it can produce more reliable estimation.

Question 2

Using the **training** data set, explore/describe the distribution of the outcome variable `survived`.

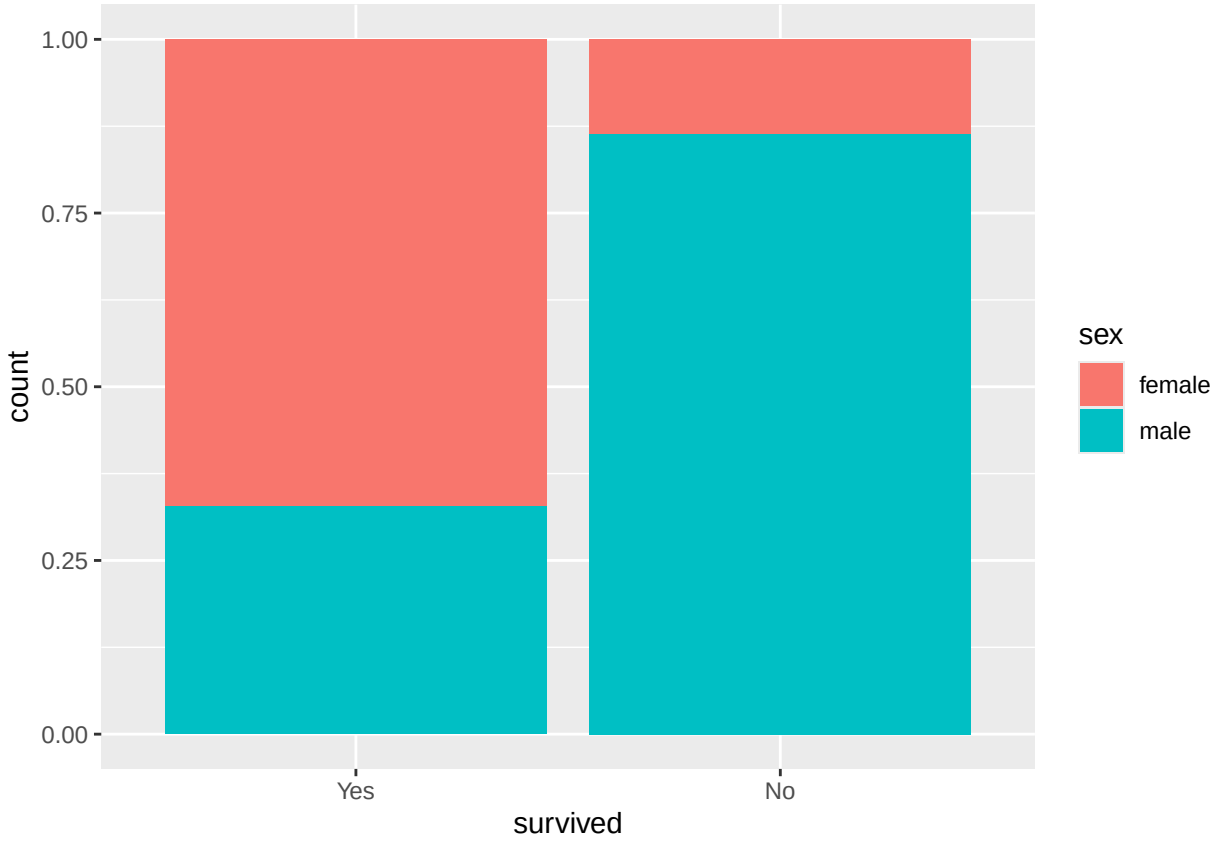
```
library(ggplot2)
train_data %>% ggplot(aes(x = survived)) + geom_bar()
```



The bar chart shows that there are more non-survivors (around 410) than survivors (around 260) in the training data, indicating that the distribution of whether surviving is very imbalanced.

Create a percent stacked bar chart (recommend using `ggplot`) with `survived` on the x-axis and `fill = sex`. Do you think `sex` will be a good predictor of the outcome?

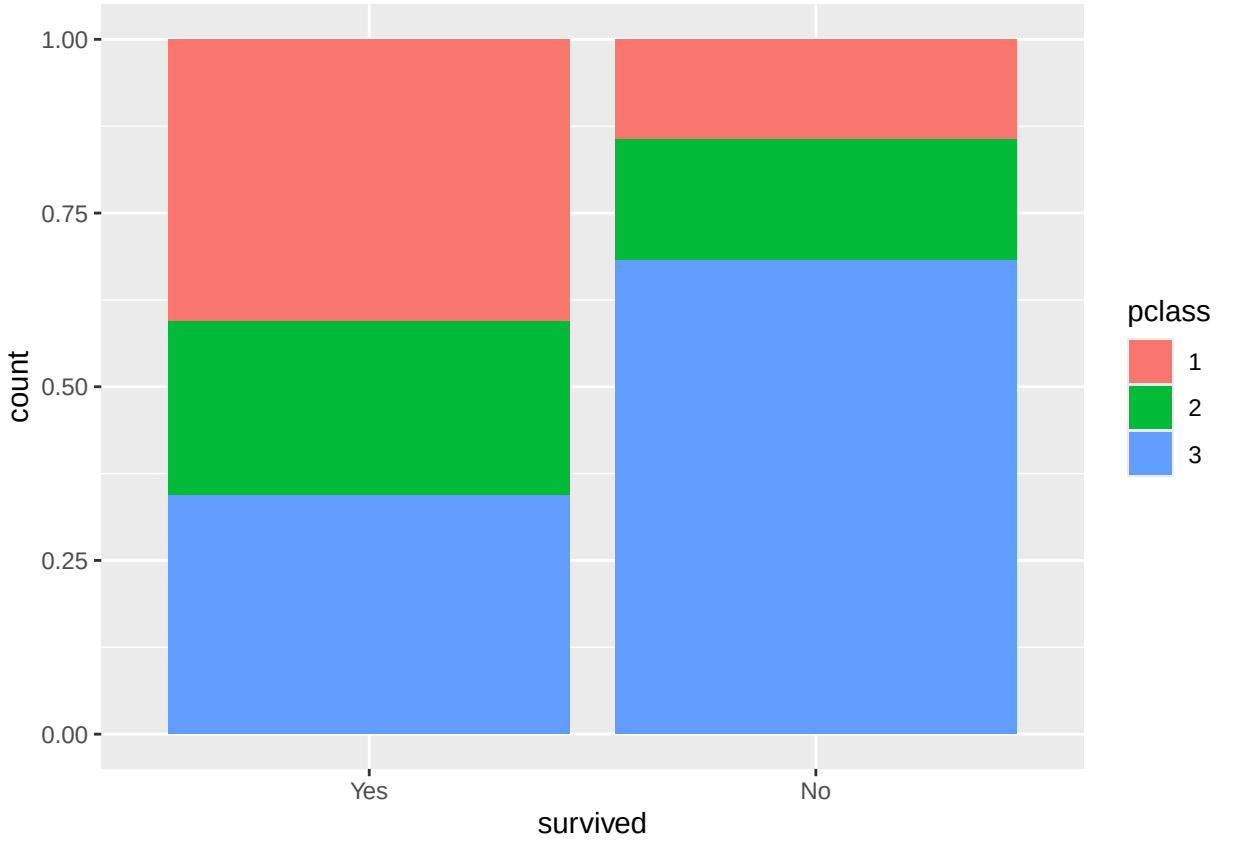
```
train_data %>% ggplot(aes(x = survived, fill = sex)) + geom_bar(position = "fill")
```



The percent stacked bar chart shows that among those who survived, a larger proportion were female, but the males accounts for a larger percentage of the non-survivors. Therefore, sex is a good predictor of the outcome because sex is strongly associated with survival.

Create one more percent stacked bar chart of `survived`, this time with `fill = pclass`. Do you think passenger class will be a good predictor of the outcome?

```
train_data %>% ggplot(aes(x = survived, fill = pclass)) + geom_bar(position = "fill")
```



The percent stacked bar chart shows that first-class passengers make up a larger proportion of survivors, while third-class passengers account for a larger proportion of non-survivors. This suggests that passenger class is associated with survival, which also appears to be a useful predictor of the outcome.

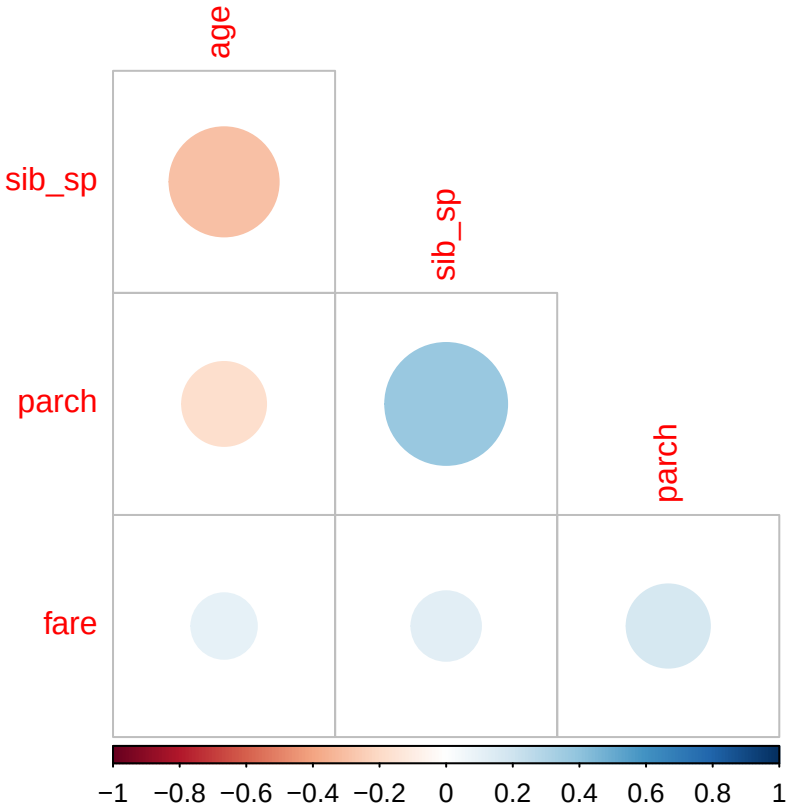
Why do you think it might be more useful to use a percent stacked bar chart as opposed to a traditional stacked bar chart?

Traditional stacked bar chart only shows counts, and this can be confusing when the group size differs. In contrast, a percent stacked bar chart makes it easier to compare proportions across groups. Since we are interested in compare the survive rate instead of the real data counts; therefore, a percent stacked bar chart allows us to visualize the relationship between predictors and outcomes more clearly.

Question 3

Using the **training** data set, create a correlation matrix of all continuous variables. Visualize the matrix and describe any patterns you see. Are any predictors correlated with each other? Which ones, and in which direction?

```
train_data %>%
  select(where(is.numeric)) %>%
  select(-passenger_id) %>%
  cor(use = "complete.obs") %>%
  corrplot(type = "lower", diag = FALSE)
```



The correlation matrix shows a moderate positive correlation between sib_sp and parch, and fare shows weak positive correlations with both sib_sp and parch. Also, age has a weak negative correlation with sib_sp. Overall, no predictors show a strong relationship with others.

Question 4

Using the **training** data, create a recipe predicting the outcome variable `survived`. Include the following predictors: ticket class, sex, age, number of siblings or spouses aboard, number of parents or children aboard, and passenger fare.

Recall that there were missing values for `age`. To deal with this, add an imputation step using `step_impute_linear()`. Next, use `step_dummy()` to **dummy** encode categorical predictors. Finally, include interactions between:

- Sex and passenger fare, and
- Age and passenger fare.

You'll need to investigate the `tidymodels` documentation to find the appropriate step functions to use.

```
titanic_recipe <- recipe(survived~pclass + sex + age + sib_sp + parch + fare, data = train_data) %>%
  step_impute_linear(age, impute_with = imp_vars(all_predictors())) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(term = ~starts_with("sex"): age + age:fare)
```

Question 5

Specify a **logistic regression** model for classification using the `"glm"` engine. Then create a workflow. Add your model and the appropriate recipe. Finally, use `fit()` to apply your workflow to the **training** data.

Hint: Make sure to store the results of `fit()`. You'll need them later on.

```
log_model <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

log_workflow <- workflow() %>%
  add_model(log_model) %>%
  add_recipe(titanic_recipe)

log_fit <- log_workflow %>%
  fit(data = train_data)
```

Question 6

Repeat Question 5, but this time specify a linear discriminant analysis model for classification using the `"MASS"` engine.

```
library(discrim)
lda_model <- discrim_linear() %>%
  set_engine("MASS") %>%
  set_mode("classification")

lda_workflow <- workflow() %>%
  add_model(lda_model) %>%
  add_recipe(titanic_recipe)

lda_fit <- lda_workflow %>%
  fit(data = train_data)
```

Question 7

Repeat Question 5, but this time specify a quadratic discriminant analysis model for classification using the `"MASS"` engine.

```
qda_model <- discrim_quad() %>%
  set_engine("MASS") %>%
  set_mode("classification")

qda_workflow <- workflow() %>%
  add_model(qda_model) %>%
  add_recipe(titanic_recipe)

qda_fit <- qda_workflow %>%
  fit(data = train_data)
```

Question 8

Repeat Question 5, but this time specify a k-nearest neighbors model for classification using the `"kknn"` engine. Choose a value for k to try.

```
knn_model <- nearest_neighbor(neighbors = 10) %>%
  set_engine("kknn") %>%
  set_mode("classification")

knn_workflow <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(titanic_recipe)

knn_fit <- knn_workflow %>%
  fit(data = train_data)
```

Question 9

Now you've fit four different models to your training data.

Use `predict()` and `bind_cols()` to generate predictions using each of these 4 models and your **training** data. Then use the metric of **area under the ROC curve** to assess the performance of each of the four models.

```
library(yardstick)

log_preds <- predict(log_fit, train_data, type = "prob") %>%
  bind_cols(train_data)

roc_auc(log_preds,
        truth = survived,
        .pred_Yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.869
```

```
lda_preds <- predict(lda_fit, train_data, type = "prob") %>%
  bind_cols(train_data)

roc_auc(lda_preds,
        truth = survived,
        .pred_Yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.865
```

```
qda_preds <- predict(qda_fit, train_data, type = "prob") %>%
  bind_cols(train_data)

roc_auc(qda_preds,
        truth = survived,
        .pred_Yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.855
```

```
knn_preds <- predict(knn_fit, train_data, type = "prob") %>%
  bind_cols(train_data)

roc_auc(knn_preds,
        truth = survived,
        .pred_Yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.966
```

```
log_auc <- roc_auc(log_preds, truth = survived, .pred_Yes)
lda_auc <- roc_auc(lda_preds, truth = survived, .pred_Yes)
qda_auc <- roc_auc(qda_preds, truth = survived, .pred_Yes)
knn_auc <- roc_auc(knn_preds, truth = survived, .pred_Yes)

results <- tibble(
  Model = c("Logistic", "LDA", "QDA", "kNN"),
  AUC = c(
    log_auc$.estimate,
    lda_auc$.estimate,
    qda_auc$.estimate,
    knn_auc$.estimate
  )
)

results
```

```
## # A tibble: 4 x 2
##   Model      AUC
##   <chr>    <dbl>
## 1 Logistic 0.869
## 2 LDA      0.865
## 3 QDA      0.855
## 4 kNN      0.966
```

Question 10

Fit all four models to your **testing** data and report the AUC of each model on the **testing** data. Which model achieved the highest AUC on the **testing** data?

```r
log_test_preds <- predict(log_fit, test_data, type = "prob") %>%
  bind_cols(test_data)

log_test_auc <- roc_auc(log_test_preds,
                        truth = survived,
                        .pred_Yes)

lda_test_preds <- predict(lda_fit, test_data, type = "prob") %>%
  bind_cols(test_data)

lda_test_auc <- roc_auc(lda_test_preds,
                        truth = survived,
                        .pred_Yes)

qda_test_preds <- predict(qda_fit, test_data, type = "prob") %>%
  bind_cols(test_data)

qda_test_auc <- roc_auc(qda_test_preds,
                        truth = survived,
                        .pred_Yes)

knn_test_preds <- predict(knn_fit, test_data, type = "prob") %>%
  bind_cols(test_data)

knn_test_auc <- roc_auc(knn_test_preds,
                        truth = survived,
                        .pred_Yes)

test_results <- tibble(
  Model = c("Logistic", "LDA", "QDA", "kNN"),
  Testing_AUC = c(
    log_test_auc$.estimate,
    lda_test_auc$.estimate,
    qda_test_auc$.estimate,
    knn_test_auc$.estimate
  )
)

test_results %>% arrange(desc(Testing_AUC))
```

```
## # A tibble: 4 x 2
##   Model    Testing_AUC
##   <chr>          <dbl>
## 1 kNN            0.851
## 2 Logistic       0.848
## 3 LDA            0.843
## 4 QDA            0.831
```

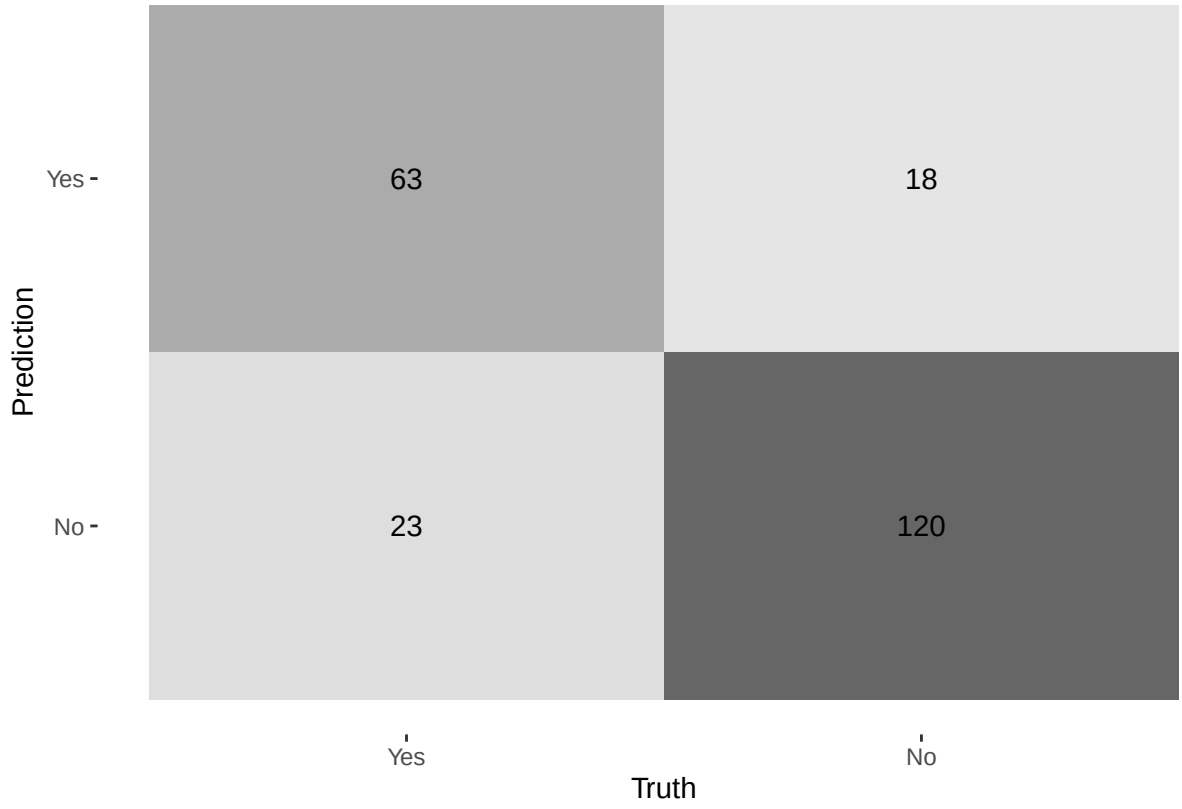
The table shown above is the result of our model performance on the testing data. Among all four models, KNN performs the best because the AUC for the testing data is 0.8511965, which is the highest, and AUC for the training data is 0.9659225, which is also the highest.

Using your top-performing model, create a confusion matrix and visualize it. Create a plot of its ROC curve.

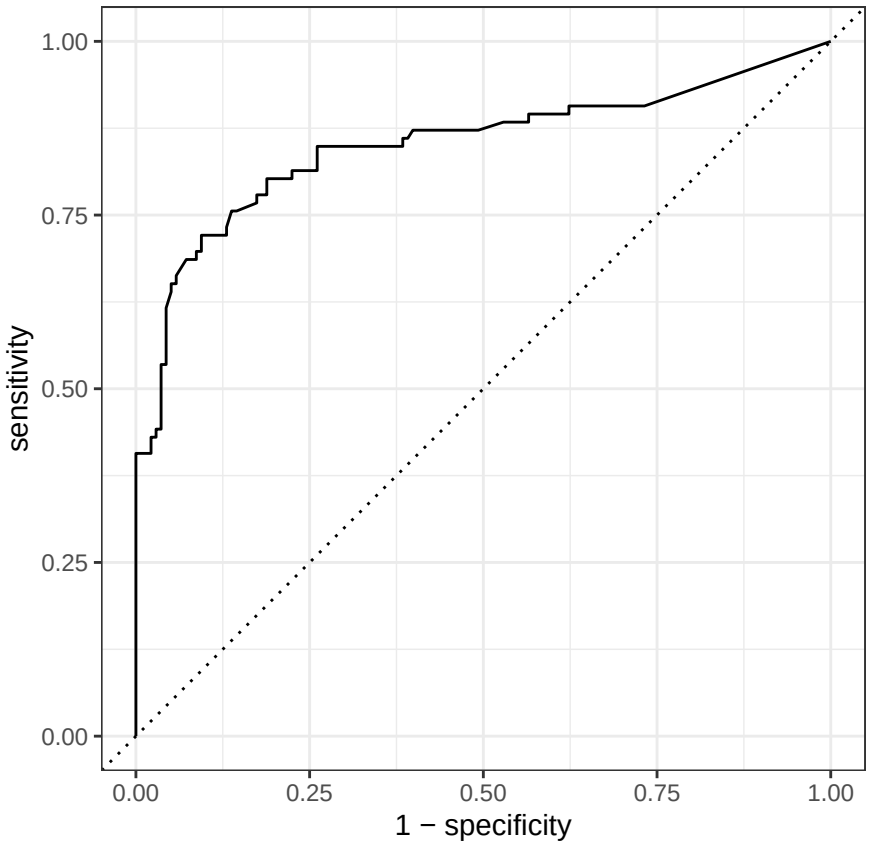
```
best_class_preds <- predict(knn_fit, test_data, type = "class") %>%
  bind_cols(test_data)

best_prob_preds <- predict(knn_fit, test_data, type = "prob") %>%
  bind_cols(test_data)

cm <- conf_mat(best_class_preds,
               truth = survived,
               estimate = .pred_class)

cm
```

```
##           Truth
## Prediction Yes  No
##        Yes  63  18
##        No   23 120
```

```
autoplot(cm, type = "heatmap")
```




```
roc_data <- roc_curve(best_prob_preds,
                      truth = survived,
                      .pred_Yes)

autoplot(roc_data)
```



How did your best model perform? Compare its **training** and **testing** AUC values. If the values differ, why do you think this is so?

The knn model achieved the highest AUC on both the training datasets, which has a value of 0.966, and testing dataset, which has a value of 0.851. However, there is a difference between the training and testing AUC values. This suggests that the knn model may be overfitting the training data. Because knn is a flexible and non-parametric model, when k is small, it can fit the training data very closely. Therefore, the performance on testing data is lower than its performance on the training data.

Required for 231 Students

In a binary classification problem, let p represent the probability of class label 1, which implies that $1 - p$ represents the probability of class label 0. The *logistic function* (also called the "inverse logit") is the cumulative distribution function of the logistic distribution, which maps a real number z to the open interval $(0, 1)$.

Question 11

Given that:

$$p(z) = \frac{e^z}{1 + e^z}$$

Prove that the inverse of a logistic function is indeed the *logit* function:

$$z(p) = ln\left(\frac{p}{1-p}\right)$$

Question 12

Assume that $z = \beta_0 + \beta_1 x_1$ and $p = logistic(z)$. How do the odds of the outcome change if you increase x_1 by two? Demonstrate this.

Assume now that β_1 is negative. What value does p approach as x_1 approaches ∞? What value does p approach as x_1 approaches $-\infty$? Demonstrate.